
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 1: Metamodelling
Project Type: Code Generation
September 25, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Introduction and Scope	1
1.1	Response to the Task 0 Feedback	1
1.2	What Is In Scope and What Is Not	1
2	Domain Analysis	1
2.1	Where the Repetition Is	1
2.2	The Concepts the Domain Requires	2
2.3	Notation	2
3	Modelling Decisions	2
3.1	Three Model Stages, Two Metamodels	2
3.2	The Layer Rule	3
3.3	Contention Decides Authority	3
3.4	Derivation Is Total, and There Is No Override	3
3.5	Values Name Model Concepts, Never Target Fields	3
3.6	Closed Vocabularies	3
3.7	Structure Carries Atomicity; the Graph Carries Order	3
4	Alternatives Considered	4
4.1	A Third Metamodel for the Generated Files	4
4.2	Grammar First or Metamodel First	4
4.3	One Resolved Model, or One Per Application	4
5	The Source Metamodel	5
5.1	The Project Document	5
5.2	The Platform Document	6
5.3	Closed Vocabularies	12
6	The Target Metamodel	13
6.1	What It Must Record	13
6.2	Structure	13
6.3	The Path Plan	14
6.4	The Generated Resources	14
6.5	No Target Field Names	19
7	The Concrete Syntax and the Constraints	19
7.1	The Concrete Syntax	19
7.2	The Constraints	19
7.3	Every Constraint Has a Fixture	21
7.4	The Second Implementation	21
8	Example Models	21

8.1	The Minimal Model	22
8.2	Refused Models	22
8.3	A Resolved Model as a Deployment View	22
8.4	Example Target Models	23
A	Worked Examples	24
A.1	The Minimal Model, End to End	24
A.2	Two Cutovers in One Project	25
A.3	A Refused Model	26
A.4	The Hand-in Archive	27

1 Introduction and Scope

Task 0 selected the code-generation problem: generate a complete Kubernetes deployment set from one abstract model of what an application needs, rather than from hand-maintained YAML in which the same fact is written down several times. This report covers Task 1, the metamodeling step. It describes the domain, the modelling decisions and the alternatives that were considered, then presents the two metamodels and the example models that instantiate them.

1.1 Response to the Task 0 Feedback

The feedback on Task 0 raised two points, and each changed something in this report.

Scope. The warning was to be careful with scope, and it was justified: the original proposal carried three metamodels and an open-ended set of generated resources. The scope is now fixed by three statements. The pipeline has *two* metamodels, not three (Section 3): the Deliverable Set is generated text, so it is not given a metamodel of its own. The generated resources are a *closed* set of six generators, listed in Section 6, and adding a seventh is a recorded decision rather than a new feature. The estate modelled is a *single* cluster, so every rule that would only become visible at a second cluster is recorded as an assumption instead of being designed for.

Notation. The question was why UML class diagrams rather than a deployment or component diagram, and whether ArchiMate or the C4 model would serve. Section 2.3 answers both: a class diagram is the notation for a language’s abstract syntax and the others describe systems rather than languages, so they cannot replace it for the metamodels; but for the example *models* a deployment view is the better picture, and Section 8 now carries one.

1.2 What Is In Scope and What Is Not

In scope	Out of scope
Two metamodels: the authored source language, and the resolved target model.	A metamodel of the generated files. They are text, and are compared as text.
A closed set of six generators for one cluster.	Additional target platforms, and a second cluster.
Validation of a source model, and the constraints that refuse an invalid one.	Applying the generated files to a cluster, which is a separate concern.
Dependency resolution between applications.	Runtime behaviour, scaling decisions and monitoring rules.

Table 1: Scope of the Project After the Task 0 Feedback

The last exclusion is the one that most affects the report’s claims. This project generates a deployment set; it does not deploy it. Every statement about correctness in this report is therefore a statement about the generated model and the generated files, never about a running cluster.

2 Domain Analysis

The domain is the deployment configuration of a small production estate: one Kubernetes cluster of seven nodes, ten namespaces, and roughly thirty long-running processes belonging to about twenty applications. The estate is real and its current configuration is hand-maintained, which is what makes it usable as evidence: every concept below was extracted from configuration that exists, and several modelling decisions are justified by a defect the existing configuration already has.

2.1 Where the Repetition Is

Four observations from the existing configuration drive the whole language.

One fact is written down many times. The hostname of one application appears in seven authoritative places across three repositories, and two conformance tests exist for no purpose other than detecting when the seven disagree. A port appears in the container specification, in the service, in the probe and in the monitoring rule. The guard was cheaper to write than the fix, which is the situation a generated deployment set removes.

One derivation is performed many times by hand. Four deployments carry the same rollout block, arrived at independently, with the reasoning recorded in comments that no tool can read. Comments record

that a recreate strategy opened a zero-pod window, and that a JVM cold start needs a startup budget an order of magnitude larger than a static bundle's. That is a rule about a class of process, expressed four times as a constant.

Values are stated in the wrong vocabulary. The existing configuration declares mechanisms, not requirements: a rollout strategy rather than whether the application may be taken down during a cutover, a node label rather than what the process needs from a machine. A process whose author forgets to declare a recreate strategy over a volume that cannot be attached twice appears to work on a one-node cluster and fails the first time a second node exists.

Some values cannot be written by one application at all. A hostname must be unique across the estate; a graphics card is held by one process at a time; the edge is a shared, finite resource. These values are contended, and no application can decide them from inside its own repository.

2.2 The Concepts the Domain Requires

The analysis produces two groups of concepts, and the split between them is the main modelling decision of the project (Section 3).

The *structural* concepts are the ones a reader would expect: a project groups applications, an application groups the processes released together, a process offers named ports and depends on the ports of others, and an application is reachable at a hostname whose paths route to its processes.

The *operational* concepts are the ones that carry most of the generated output: what a process needs from a machine, what its data is worth if it is lost, which secrets it may read and how it receives them, what answers a health check, how long a cold start may legitimately take, and whether a new version may run beside the old one during a cutover. These have no counterpart in a structural architecture notation, and they are where the interesting derivations live.

2.3 Notation

The Task 0 feedback asked why UML class diagrams are used, and suggested ArchiMate or the C4 model as alternatives or as additions.

For the metamodels themselves the class diagram is not really a choice. A metamodel defines a language's abstract syntax, Ecore implements EMOF, and EMOF is classes, attributes, references and multiplicities, so a class diagram is the notation whose constructs correspond one-to-one to what the metamodel is built from. ArchiMate and C4 describe a system, not a language, so neither can define one.

For the *models* the suggestion is a good one, and this report takes it up. A resolved model is an arrangement of processes on machines, which is exactly what a deployment view is for, and the resolved model contains every fact such a view needs: which process runs where, which application answers on which hostname, and which process depends on which. Section 8 shows one example model that way rather than only as text, and generating an ArchiMate technology view or a C4 deployment diagram from a resolved model is a second model-to-text transformation over the same target metamodel, which Task 3 could add at little cost.

3 Modelling Decisions

Seven decisions shape both metamodels. Each is stated with the observation that forced it, because in every case the alternative had already been tried in the estate's hand-maintained configuration.

3.1 Three Model Stages, Two Metamodels

The pipeline has three stages: an authored model, a resolved model, and the generated text. Only the first two are given metamodels. The generated files are text in a target syntax that already has a schema of its own, so giving them a third metamodel would restate that schema without adding a constraint the project could check.

The three stages are deliberately *not* described as metalevels. No stage is a type model of the stage below it, so calling the arrangement a metamodel hierarchy would claim an `instanceOf` relation that does not hold. They are stages of one transformation pipeline, and the word metamodel is reserved for a language definition.

3.2 The Layer Rule

The rule that makes the middle stage worth having: **the authored model contains no mechanisms, and the generated text contains no decisions.** Every value is assignable to exactly one stage by two questions. Must a human author it? And does it record a decision, or merely serialise one?

The counter-experiment is on record. An earlier two-stage arrangement, with resolution private to the generator, produced three mutually incompatible documents all claiming the same version identifier, because a two-stage vocabulary could not say which of the three was the authoring shape. The middle model exists so that a decision has one place to live and one name.

3.3 Contention Decides Authority

A value is decided by the platform if and only if it must be unique across the estate or draws on a shared finite resource; every other value is declared by the application and carried through untouched. One question replaces a per-field negotiation.

The rule required one refinement that is worth reporting, because the naive reading is wrong. Contention decides who *arbitrates*, not who *authors*. Memory and CPU are draws on a finite pool, so a who-authors reading would forbid an application from stating them, which leaves every process with no reservation at all. The application therefore states its requirement and the platform arbitrates it against what the machines publish. The accepted cost is named rather than hidden: nothing today stops an author overstating the requirement, because the arbitration is an eligibility test against one machine rather than a sum across the estate.

3.4 Derivation Is Total, and There Is No Override

Every value the platform decides is a function of the authored model and the platform's own facts. A derived value has exactly one declaring site, so the metamodel carries no override mechanism and no exception vocabulary.

This decision was taken against evidence that initially argued for a hatch: one process carried a hand-set deadline three times smaller than its neighbours, justified by the observation that it starts in seconds rather than minutes. That is not a value only its owner could know; it is evidence that one rule over the startup budget was wrong for a whole class of process. The correct response is a rule that reads an input the process already declares. Keeping the hatch would have hidden a wrong derivation behind a per-process reason, where removing it makes the same error visible as a wrong result for the entire class.

3.5 Values Name Model Concepts, Never Target Fields

No attribute of the source metamodel, and no attribute of the target metamodel, is named after a field of the generated syntax. A process declares whether it may be taken down during a cutover, not a rollout strategy; it declares the paths it must write, not a security context. The target metamodel records the platform's decisions in the same vocabulary, and the generators are the only place a target field name is spelled.

This is what keeps the target metamodel a model rather than an object tree with extra steps, and it is checked directly: no key of either metamodel may be a field name of the target syntax.

3.6 Closed Vocabularies

Wherever an attribute admits a fixed set of values, the set is closed in the metamodel rather than typed as a string. The estate supplies the argument: one existing field was typed as a free-form string, every application wrote the same value into it, and not one of those values was correct. A field that is wrong in every observed case at no cost is a comment, not a declaration, and closing the vocabulary is what turns it back into one.

3.7 Structure Carries Atomicity; the Graph Carries Order

Two groupings are needed, and deriving one from the other was tried and rejected. What must be released together is *structural*: the processes of one application switch together, or none of them switches. What must be applied before what is *derived* from the dependency graph. Deriving atomicity from the graph instead takes the transitive closure of every edge and turns the whole estate into one unit, which would make every change estate-wide.

4 Alternatives Considered

Table 2 lists the alternatives encountered while building the metamodels, with the reason each was rejected. The three discussed in prose below are the ones where the rejection is not obvious.

Alternative	Why it was rejected
Template the target files directly (Helm, Kustomize overlays)	No abstract model, so no constraint can be written and no fact can be derived once; the repetition the project exists to remove stays.
Annotate the target files with extra fields	Tried in the estate: the authoring shape and the resolved shape end up sharing one name, and no vocabulary can say which document is wrong.
Two stages, resolution private to the generator	The generator then takes decisions that appear in no model and can be reviewed nowhere.
A third metamodel for the generated files	The target syntax already has a schema; a third metamodel restates it.
A grammar first, with the metamodel inferred	Puts the concrete syntax in charge of the concepts; the same language has to be readable from two syntaxes.
Free-form strings instead of closed vocabularies	Tried in the estate: the one free-form field was wrong in every observed case.
An override mechanism for derived values	Hides a wrong derivation behind a per-process reason instead of exposing it for the whole class.

Table 2: Alternatives Encountered and the Reason Each Was Rejected

4.1 A Third Metamodel for the Generated Files

The Task 0 proposal carried three metamodels, one per stage of the pipeline. It was reduced to two, and this is the clearest scope reduction the feedback produced.

The argument for a third metamodel is that it would let the generation step be checked structurally rather than as text. The argument against it is that the target syntax is already schema-defined, so the third metamodel would be a hand-maintained restatement of a schema that exists, with a drift problem of its own and no constraint that the second metamodel cannot already express. The generated files are therefore compared as text against reviewed expected output, and the typed target resources live inside the second metamodel, where the generation templates can walk them.

4.2 Grammar First or Metamodel First

The source language has a textual concrete syntax, so the language could have been defined by a grammar with the metamodel inferred from it. It was defined the other way round: the metamodel first, with the grammar as one syntax over it.

The reason is that the same source model must be readable from two concrete syntaxes, and the production implementation parses a different serialisation of the same language. If the grammar were in charge, the two would agree only by coincidence. With the metamodel in charge, both syntaxes are checked against one structure, and that structure is exported as a machine-readable description that both implementations are held to.

4.3 One Resolved Model, or One Per Application

The resolved model covers the whole estate in one document, rather than being produced per application. Some of what it records cannot be computed for one application alone: which shared edge carries a hostname, the order in which units are applied, and what an application derives from the applications that depend on *it*. Producing a per-application model directly would either recompute those properties inconsistently or omit them.

What an application’s own repository receives is therefore a *projection*: the slice belonging to that application, obtained by filtering the estate-wide model rather than by a separate computation, so the two cannot disagree about what was decided.

5 The Source Metamodel

The source language is authored in two documents that together form one metamodel. A *project* document is written by the owner of a group of applications and states what those applications need. A single *platform* document is written once for the estate and states what the estate offers. The split follows directly from the contention rule of Section 3: a value that must be unique across the estate, or that draws on a shared finite resource, is stated in the platform document, and everything else is stated by the application that needs it.

5.1 The Project Document

Figure 1 shows the project half of the source metamodel. Its root is a **Project**, which carries an owner and the applications it groups. An **Application** is the unit that is released as a whole: it carries an identifier that is unique across the estate, the processes released together, the hostnames it is reachable at, the secrets it holds, how its database schema moves when a new version replaces the old one, and, optionally, how its failures should be alerted.

A **Migration** is a union of three answers. A **ManagedMigration** names a Liquibase changelog, by its path in the application’s repository, that the platform’s runner applies before any new version starts. A **SelfMigration** says the image migrates its own schema, which is the only honest answer for a third-party image. A **NoMigration** says the application moves no schema, because another application of the same project does. The answer is required exactly where an application reaches a database, and one application of a project moves that project’s one database.

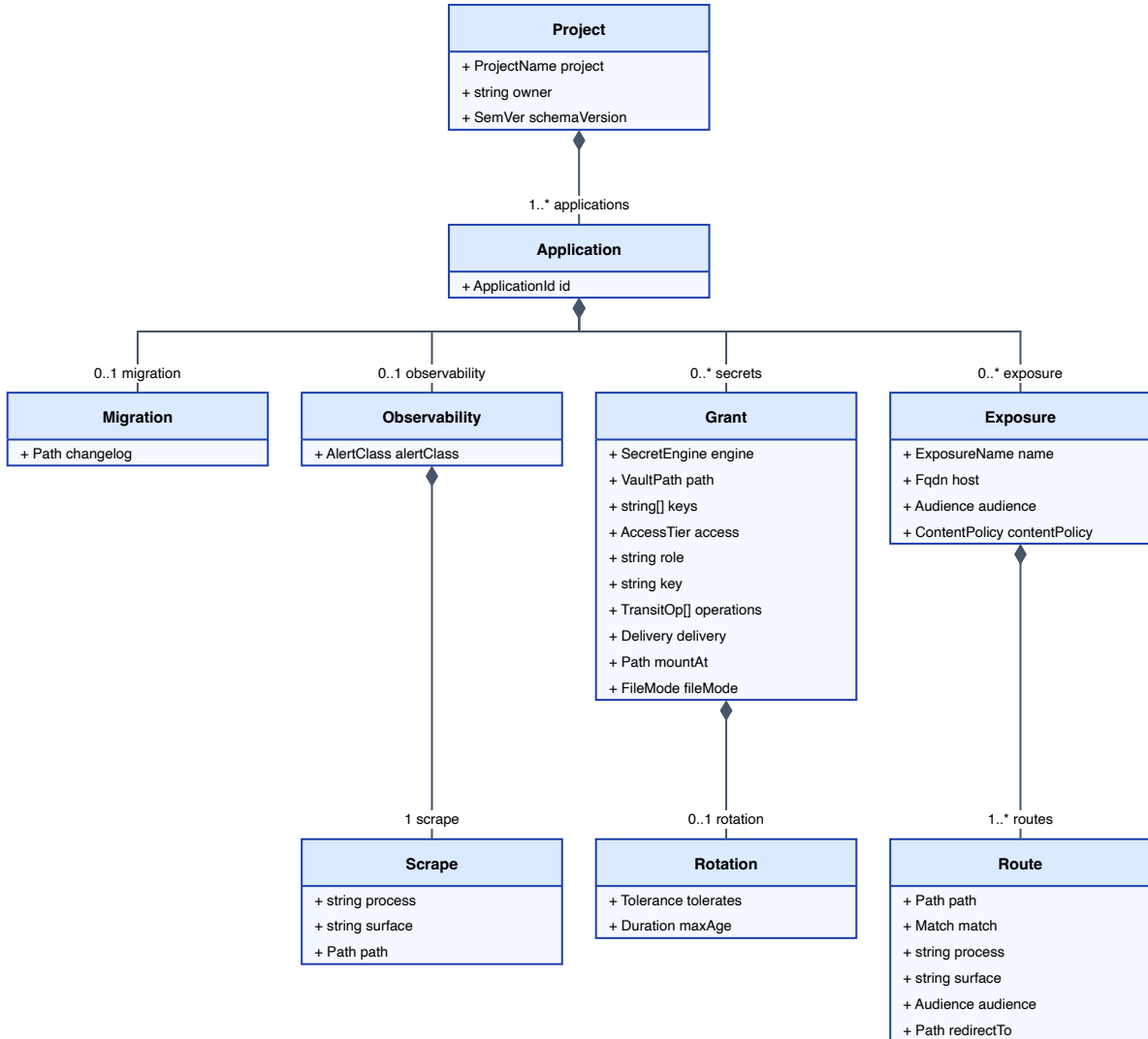


Figure 1: The Project Document: the Application and What Belongs to It

A **Process** is the level at which almost every operational concern sits. It names the image it runs, what kind of thing it is, which named ports it offers, what it needs from a machine, which paths it must write, what answers a health check, what its volumes are worth, and whether it must keep serving while a new version replaces it. That last answer is its *cutover*: **continuous** asks for the new version to start beside the old one and take traffic only once every process of the application has passed analysis, and **interrupted** accepts a stop-then-start. A process's kind is one of **application**, **job**, or **prepare**: a prepare process is forward-only setup, such as a seed, that runs to completion after the migration and before the new version starts, and serves nothing. The concerns that belong to the application rather than to one of its processes are kept on the application: a hostname fronts several processes and picks between them by path, so exposure cannot sit on a process without being written twice.

A **Process** carries almost every operational concern, so it is shown in four figures rather than one: Figure 2 its capacity, helpers and health, Figure 3 what it needs from a machine, Figure 4 its storage and files, and Figure 5 its configuration, dependencies and the ports it offers. All four are cut from the same source as Figure 1.

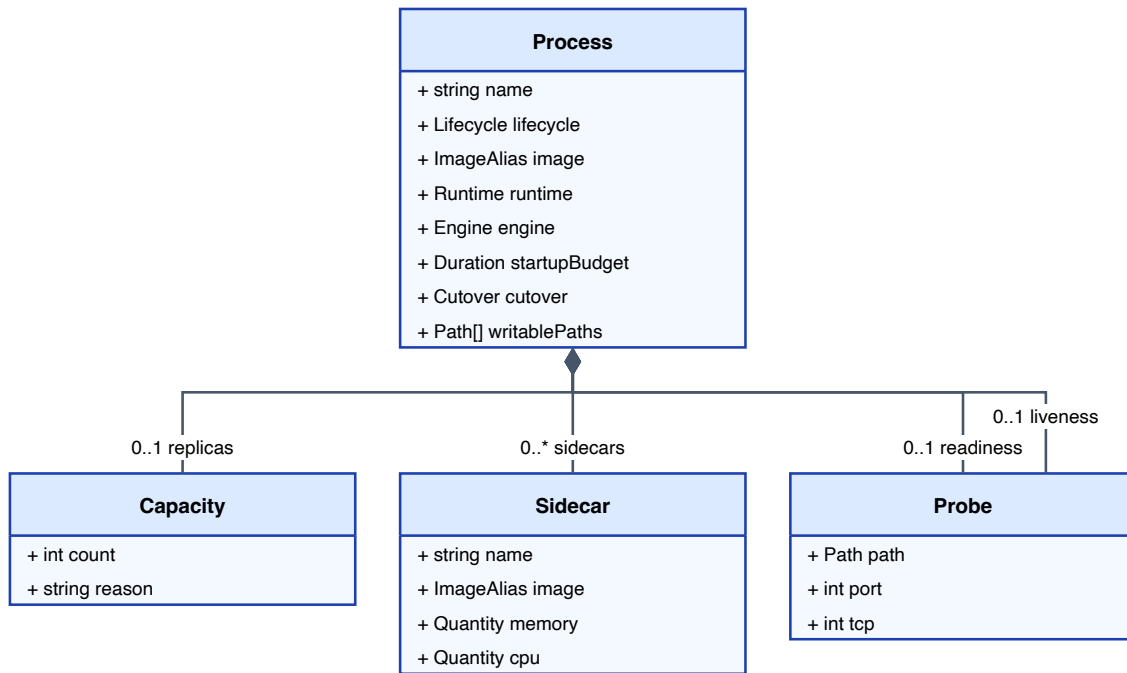


Figure 2: A Process: Capacity, Helpers and Health

Table 3 lists the classes and what each one is for.

5.2 The Platform Document

The platform document carries the facts and policies that no single application may decide: what the cluster is, where the edge terminates and which audiences each edge tier serves, one backup policy per durability class, one backup method per engine, one hardening posture, one probe cadence, one monitoring cadence, the services the estate runs without deploying them, the runner every managed migration builds on, how a release is switched: which applications perform the switch and are never switched by it, and the cadence at which a new version is analysed, where each project's rendered output is published and whose signature the cluster accepts on it, and, while the estate moves off its old delivery path, which path each project is on.

It is shown in four figures, cut from its own source by the same script that cuts the project half. Figure 6 holds the facts about the cluster and what must exist before a rendered object can land, including where the reconciler fetches each project's rendered output and whose keyless signature it verifies, Figure 7 the edge and what the estate already runs, and Figure 8 the policies: one per durability class, one per engine, and the cadences the derivations read. Figure 9 holds what delivery reads: the migration runner and its terms, the delivery machinery with its analysis cadence, and the handover ledger.

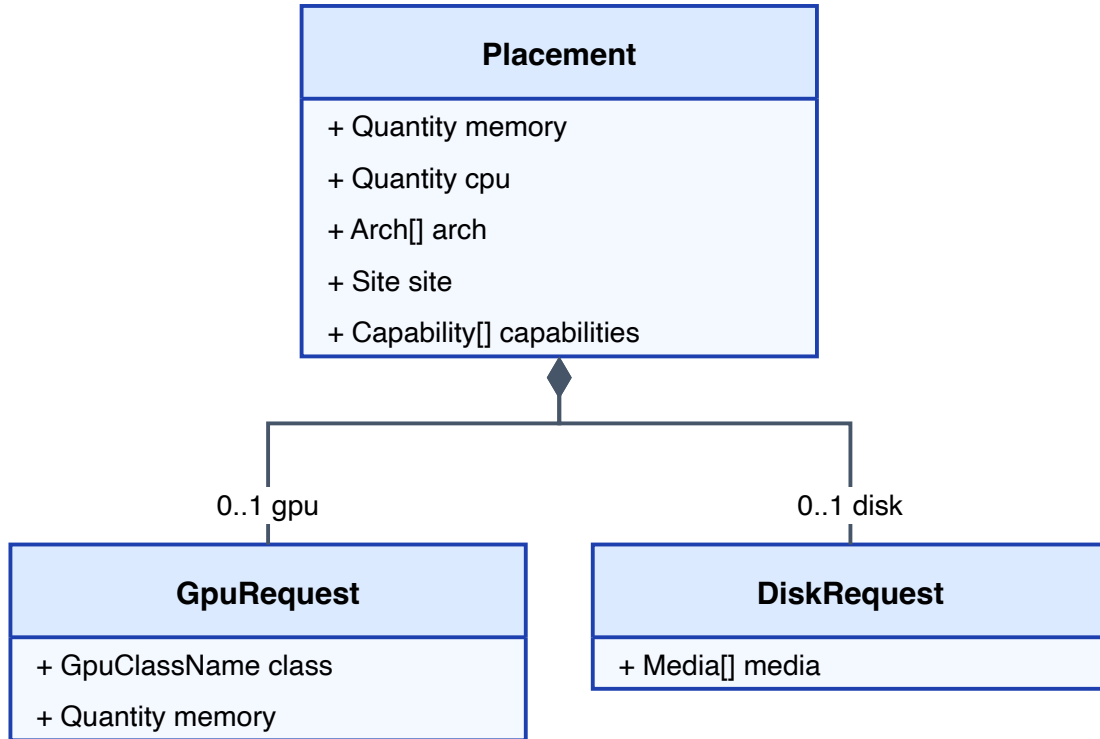


Figure 3: A Process: What It Needs From a Machine

Class	What it represents
Project	The authored unit: an owner and the applications the document groups.
Application	What is released as a whole, identified uniquely across the estate.
Process	One containerised process: image, kind, ports, needs and health.
Surface	A named port a process offers, declared once.
Exposure	A hostname the application is reachable at, and who may reach it.
Route	A path on that hostname, and the process and surface that serve it.
Placement	What a process needs from a machine, as hard requirements.
Volume	Storage a process holds, with what its data is worth if lost.
Probe	What answers readiness or liveness for a process.
Dependency	An edge to another application's named surface, carrying, where the provider owns databases, how the credential it derives may rotate.
Credentials	The rotation an application can honour for the database credential its edge derives.
Migration	How the application's schema moves: a Liquibase changelog the platform applies, the image itself, or nothing.
Grant	A secret a process may read, how it is delivered, and its rotation.
Asset	A file a process needs that is configuration rather than code.
Observability	Where metrics are collected and how urgently failures alert.
Sidcar	A helper process sharing the main process's lifetime.
Capacity	A count above one, with the reason it is not derived.

Table 3: Classes of the Project Document

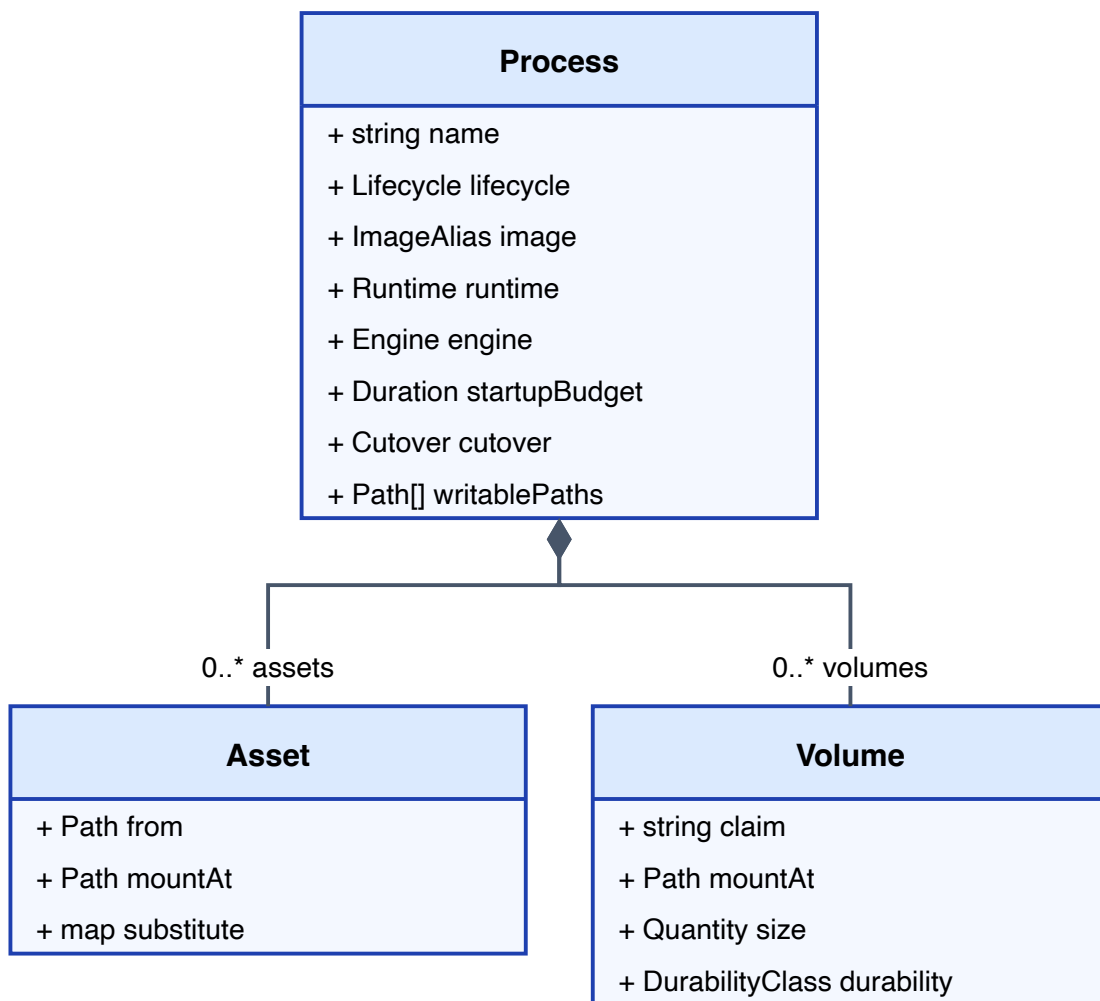


Figure 4: A Process: Storage and Files

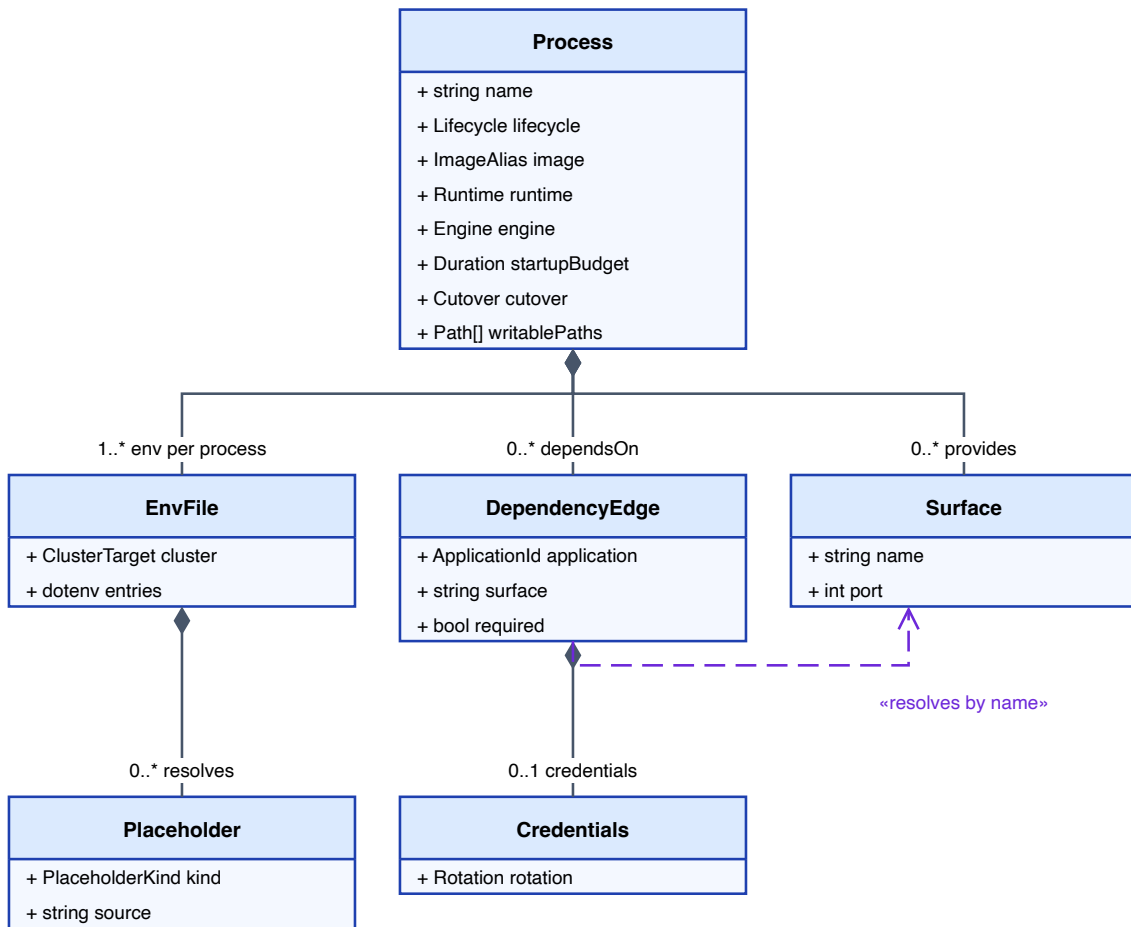


Figure 5: A Process: Configuration, Dependencies and the Ports It Offers

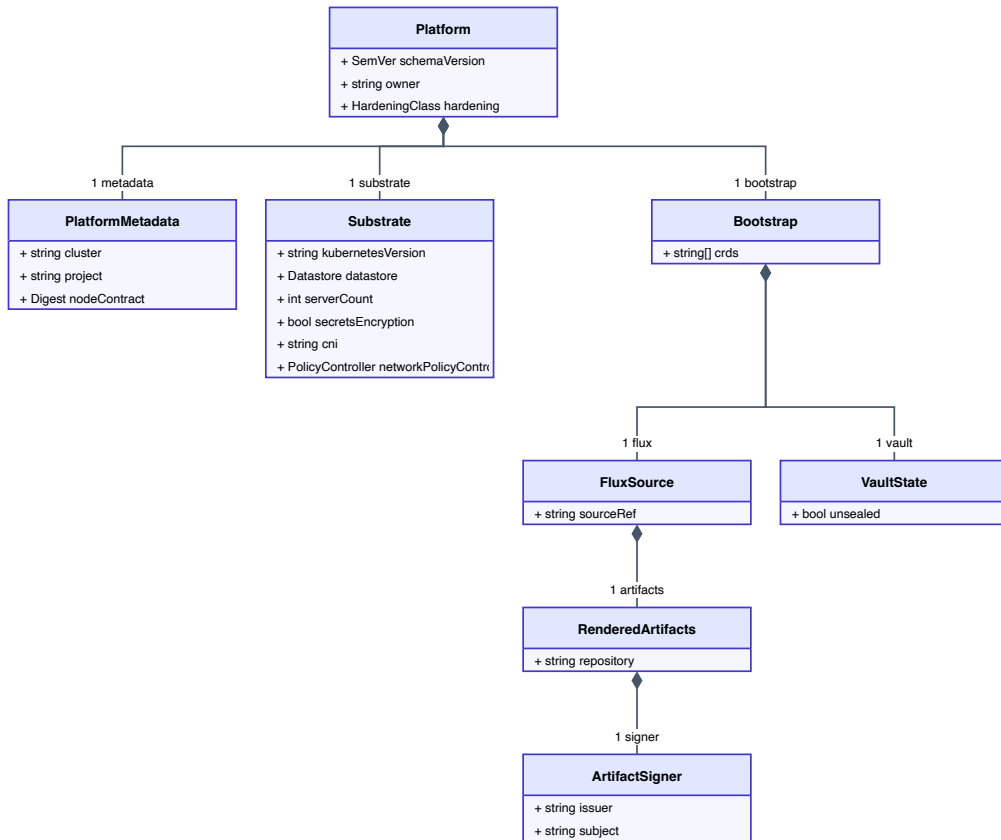


Figure 6: The Platform Document: the Cluster and What Must Already Exist

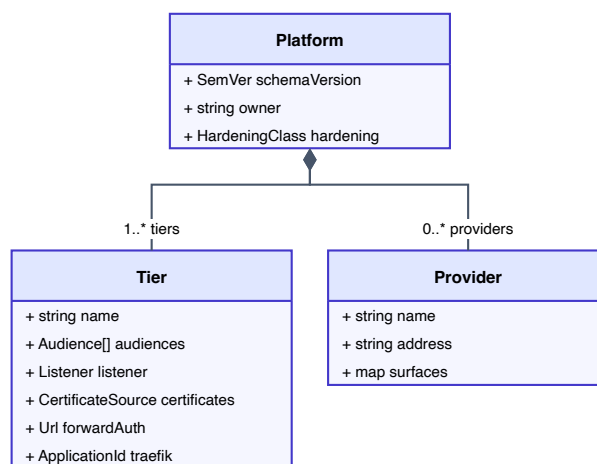


Figure 7: The Platform Document: the Edge, and What the Estate Already Runs

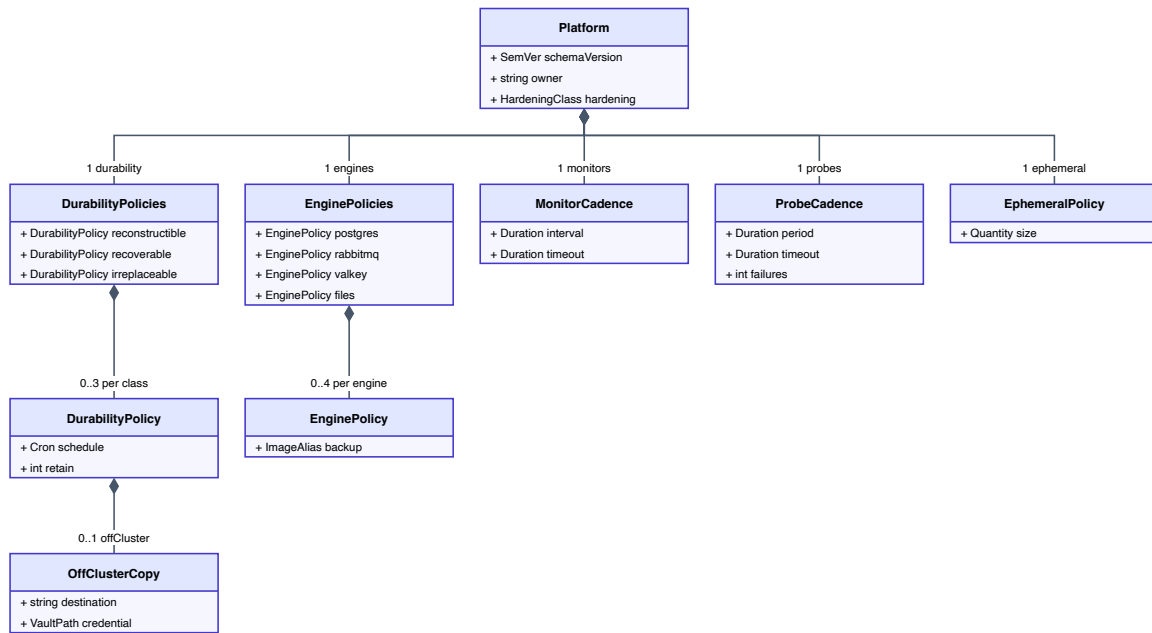


Figure 8: The Platform Document: the Policies a Derivation Reads

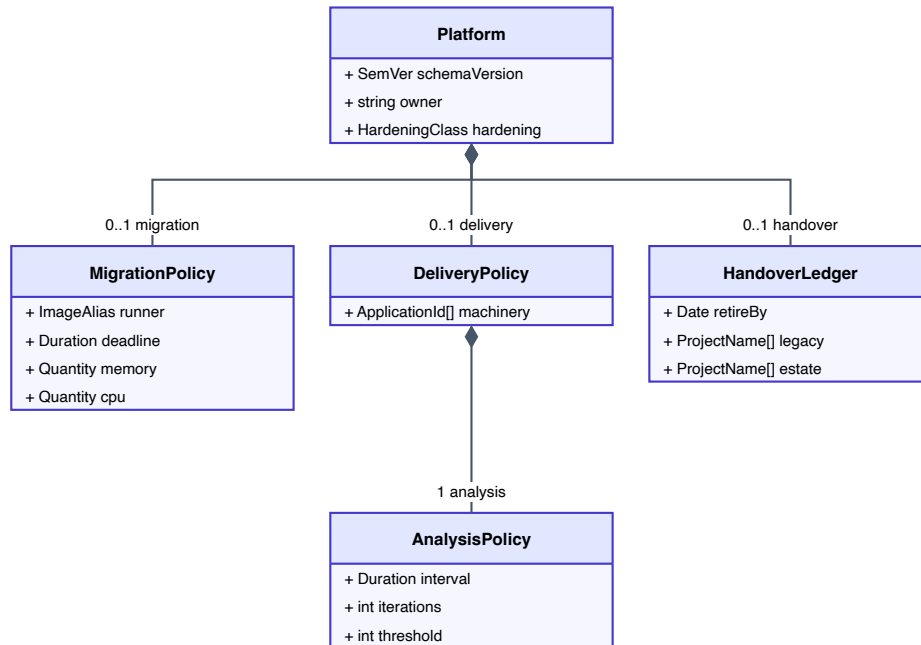


Figure 9: The Platform Document: What Delivery Reads

A fact here is named for what it is and never for how the cluster is told about it, which is the rule of Section 3 applied to the document that is most tempted to break it. Two of these facts gate a constraint rather than feed a derivation: a cluster that does not encrypt secrets at rest refuses every delivery that would write one into it, and a durability class or an engine with no policy refuses the volume or the process that asked for it (Section 7).

Table 4 lists the classes.

Class	What it represents
Platform	The estate-wide authored document: one per cluster, with its owner and hardening posture.
PlatformMetadata	Which cluster and project the document describes, and the digest of the machine contract it was written against.
Substrate	Facts about the cluster that decide other decisions: its version, datastore, server count, whether secrets are encrypted at rest, and its network plugin and policy controller.
Bootstrap	What must already exist before the first rendered object can land, recorded rather than declared.
FluxSource	The reconciler's source reference, part of that bootstrap set.
RenderedArtifacts	The registry repository each project's rendered output is published to, one signed artifact per project.
ArtifactSigner	The keyless identity, an issuer and a subject, whose signature the reconciler accepts on a rendered artifact.
VaultState	Whether the secret store is unsealed, the second bootstrap fact.
Tier	Where the edge terminates: the audiences it serves, its listener, its certificate source, the endpoint that decides an authenticated request, and the declared Application that is its proxy.
DurabilityPolicies	The three durability classes' policies, held together so a missing one is visible.
DurabilityPolicy	One class's backup window and retention.
OffClusterCopy	Where an irreplaceable volume's copy goes, and the credential that writes it.
EnginePolicies	The backup method per engine, held together for the same reason.
EnginePolicy	One engine's backup image alias, named and never executable.
MonitorCadence	How often the estate collects metrics, and how long it waits.
ProbeCadence	The estate's one probe cadence: period, timeout and the failures tolerated.
EphemeralPolicy	The size a declared writable path receives.
Provider	Something the estate runs and this model does not deploy, with its address and named surfaces, which a dependency may resolve against.
MigrationPolicy	The runner image alias every managed migration builds on, with the deadline and resources its run takes.
DeliveryPolicy	The applications that perform a switch and are never switched by one, linked across documents like a tier's proxy.
AnalysisPolicy	How often a new version is checked, how many passing checks promote it, and how many failing ones roll it back.
HandoverLedger	While the estate moves off its old delivery path: the projects still delivered by it, the projects delivered by their pin, and the date the old path is removed.

Table 4: Classes of the Platform Document

5.3 Closed Vocabularies

Wherever an attribute admits a fixed set of values, the metamodel declares an enumeration. The project document contributes vocabularies for the kind of a process (**application**, **job**, **prepare**), its runtime, its engine, its cutover (**continuous** or **interrupted**), a volume's durability class, machine architecture and storage medium, alert urgency, the audience of an exposure, a content policy, a path match, a secret's access tier and delivery, and a rotation tolerance. The platform document contributes vocabularies for the cluster's datastore and policy controller, an edge tier's listener and certificate source, and the hardening class.

The enumerations are the surface most of the constraints are written against, which is the argument of Section 4 against free-form properties: a constraint can enumerate an enumeration exhaustively, and cannot enumerate a string.

6 The Target Metamodel

The target metamodel is the language the model-to-model transformation of Task 2 writes into, and the language the code generation of Task 3 reads. It holds every decision the platform took, recorded in the same vocabulary as the source metamodel, together with the typed target resources and the output path of each one.

6.1 What It Must Record

A resolved model is complete in a specific sense: the generators may read it and nothing else. Anything a generator would otherwise have to decide for itself is recorded here, because a decision taken while serialising appears in no model, is reviewed nowhere, and is invisible in the projection an application's owner reads back.

Three groups of content follow from that.

Decisions. Which namespace an application lives in; which edge tier carries each hostname and what the middleware chain in front of it is; the precedence of each route; the eligible machines for each process and the machine a bound volume already ties it to; the identity of each process and the secret paths it may read; the resolved image digest; the derived probe timings, the deadline, the instance count and the capacity; the backup plan each volume's durability class earns; the resolved dependency edges with the policy peers they permit; the *switchover* each process's cutover derives (**blue-green**, **stop-start**, or **rolling** for the applications that perform a switch); and, where an application moves its schema with a changelog, the migration image its changelog was built into and the serving revision its compatibility was proven against.

Order and atomicity. The units in which the estate is applied, with the edges that say which precedes which, and, per application that switches blue/green, the inputs of the gate that decides when a new version may receive traffic: its members, each member's readiness and the checks it is analysed on, the analysis cadence, and the deadline. A release gate holds every member at a barrier until all have passed, so the rule that an application switches as one is decided here and performed by delivery.

Identity of a release. Each application's *revision*: the digest of its own element of the resolved model, the revision itself excluded. It moves exactly when a decision about that application moves, and not when some other application's does, which is what lets a migration, a gate and a held release all name one release of one application.

Provenance. The hash identifying the render, and one digest per pinned input: every project document, the platform document, the machine contract, the image lock and the observed-state snapshot. This is what makes reproducibility a checkable claim rather than an assertion: identical recorded digests must produce identical output, so a difference under identical digests is a defect in the generator.

6.2 Structure

The metamodel is shown in five figures, each cut from one source by the same script that cuts the source metamodel's figures.

The root is a **ResolvedDeployment** covering the whole estate, because several of its properties are estate-wide and cannot be computed for one application alone. It carries the provenance, the resolved applications, the units and their ordering, and the path plan.

A **ResolvedApplication** is both the element the estate-wide model contains and the document published back to an application's own repository. In the second role it carries its own provenance; nested in the estate-wide model it does not, because the enclosing document's provenance already covers it. This is what makes the projection a filtering of the estate-wide model rather than a second computation, so the two cannot disagree.

A **ReleaseGate** carries its **GateAnalysis** cadence and one **GateMember** per blue/green process, and is absent wherever no process switches blue/green: an interrupted application stops before it starts, and an application of the delivery machinery rolls in place, so nothing waits on a gate. A **ResolvedMigration** records only what varies: the migration image, the revision it was tested against, and whether the release holds a changeset that cannot run in a transaction, which no automatic undo may touch; the identity that runs it, the owner role and the deadline are fixed functions of the application and the platform document, so recording them would repeat a derivation.

Below it, a **ResolvedProcess** carries the per-process decisions; its cutover and switchover are both unset on a prepare process, which runs once and cuts over nothing, a **ResolvedExposure** carries the hostname with its tier, middleware chain and ordered routes, and the volumes, grants, dependency edges and writable paths each

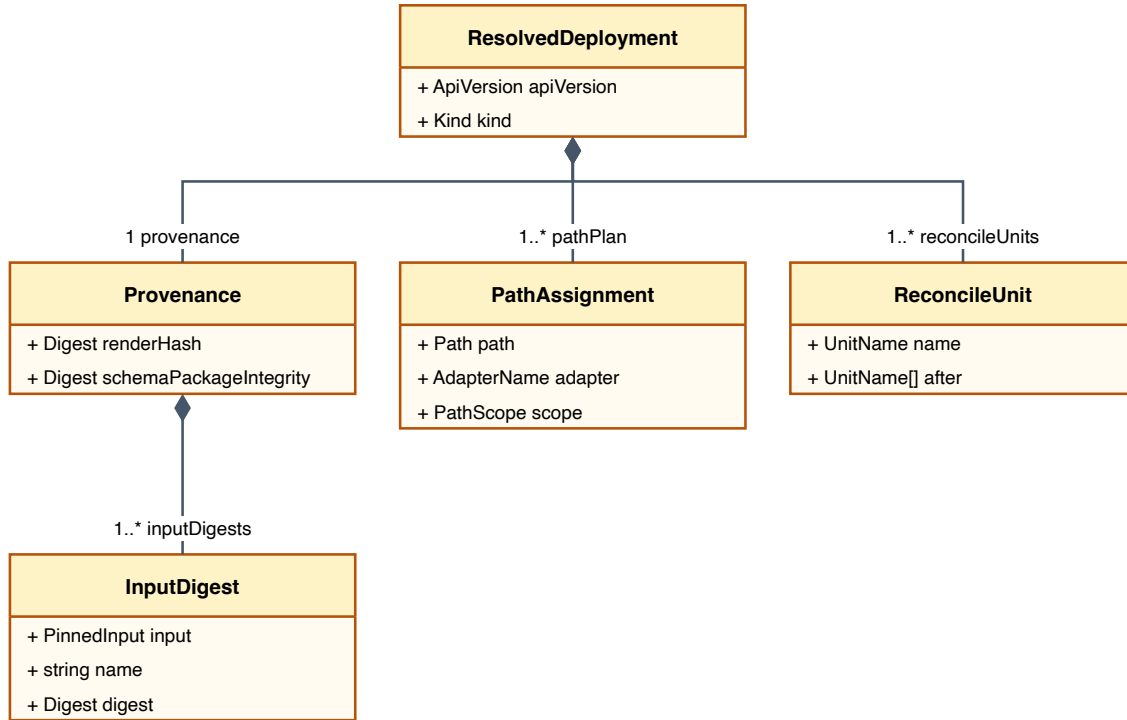


Figure 10: The Estate-Wide Frame: Provenance, the Path Plan and the Units

carry what the platform decided about them.

6.3 The Path Plan

Every output path is assigned here, not by the generator that writes the file. The model carries, for each object to be generated, the generator that owns it and the path it is written to.

The rule follows from the layer rule rather than adding to it, and two observed cases show that the alternative does not work. An object that belongs to a group of applications rather than to one of them is emitted once per application when the path is computed per generator, producing several identical objects at several paths. And an estate-wide object lands in whichever namespace the generator that emitted it happened to key off. Assigning the paths in the model also moves the check that two generators never claim one path to a point where the complete set of paths is known, which is before any generator runs.

6.4 The Generated Resources

The generation step has a closed set of six generators, one per subsystem: the workloads and their supporting objects, including the Canary through which a blue/green process is switched, the network policy, the monitoring objects, the edge routing, the secret-store policies, and the secret delivery objects. The set being closed is the scope commitment of Section 1: adding a seventh is a recorded decision, not a new feature.

The metamodel for the code. The task asks for a metamodel of the code the project generates. Here the generated code is a set of Kubernetes, Flux, Flagger, Traefik and Vault files, and this metamodel describes it in two halves. The resolved decisions say *what* each file must contain. One typed class per resource family (`WorkloadFile`, `SwitchoverFile`, `NetworkPolicyFile`, `MonitorFile`, `IngressRouteFile`, `SecretFile`, `IdentityFile`, `NamespaceFile` and `IndexFile`) says *which* file is generated, at which path, by which generator. The files' own schemas are not modelled a third time: they are published by their projects, the generated tree is validated against them, and modelling them again would copy five upstream vocabularies into this project with nothing to keep them in step. A model-to-text template therefore reads one instance of this metamodel and writes one file per typed resource, deciding nothing.

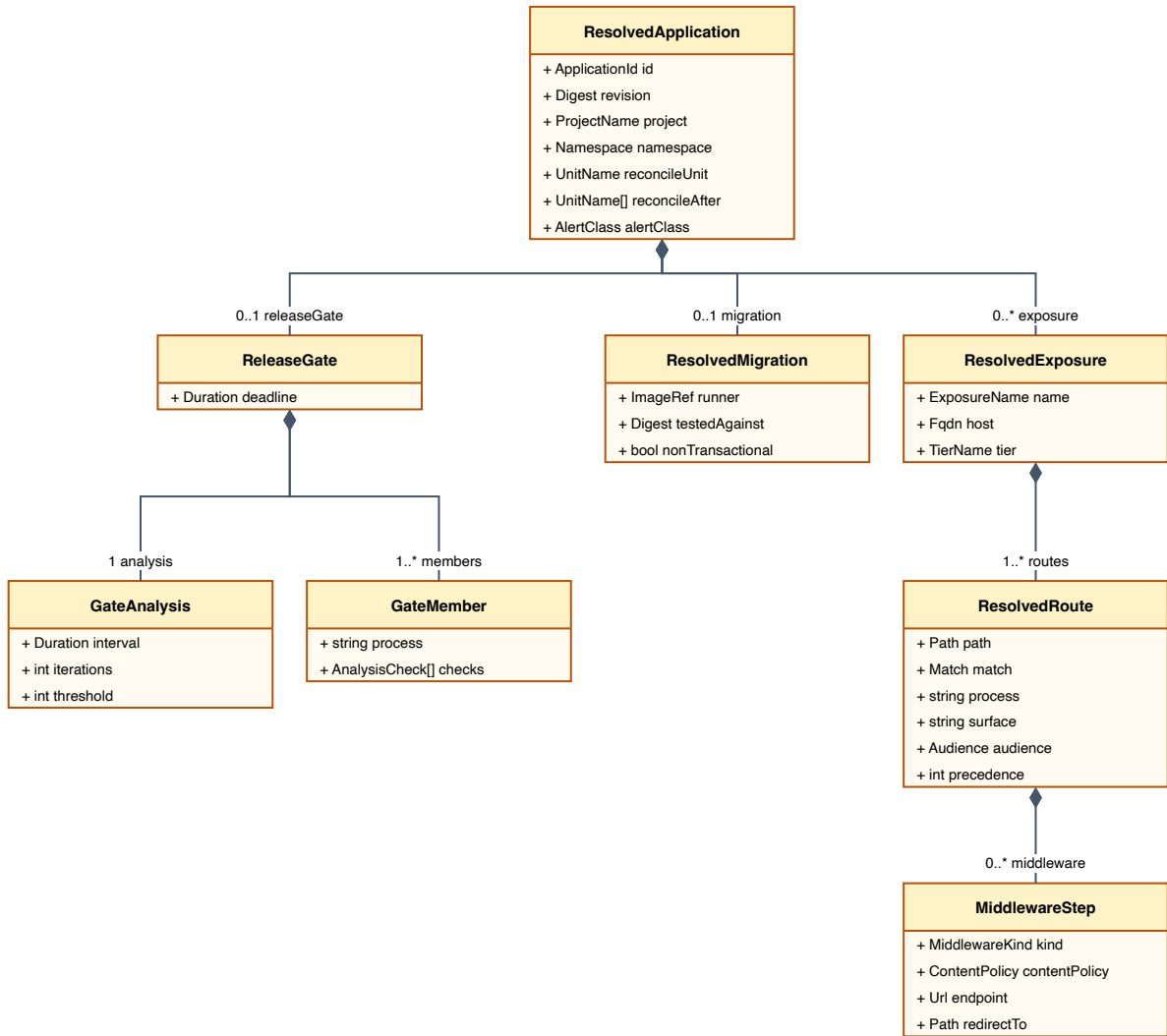


Figure 11: A Resolved Application: Its Release Gate, Its Migration and Its Exposure

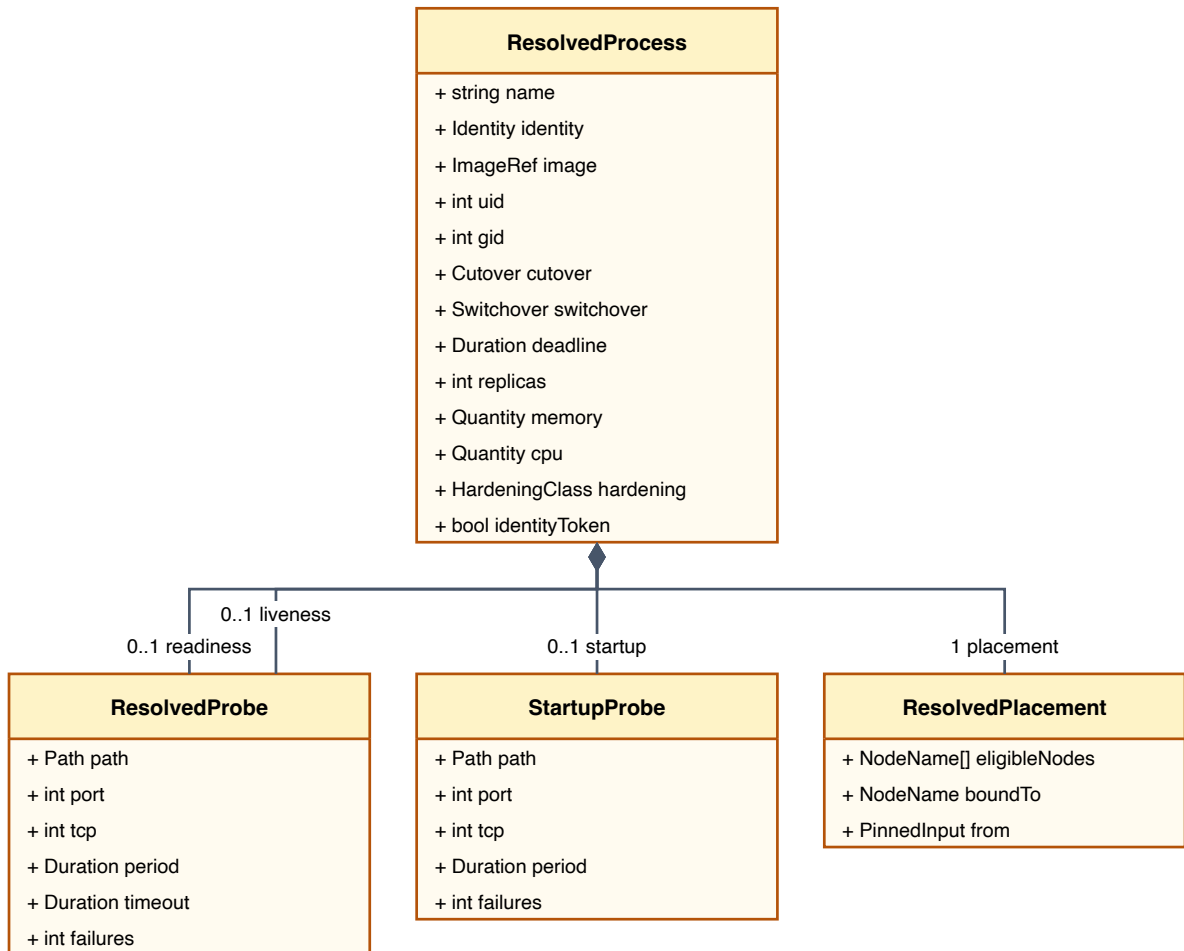


Figure 12: A Resolved Process: Health and Placement

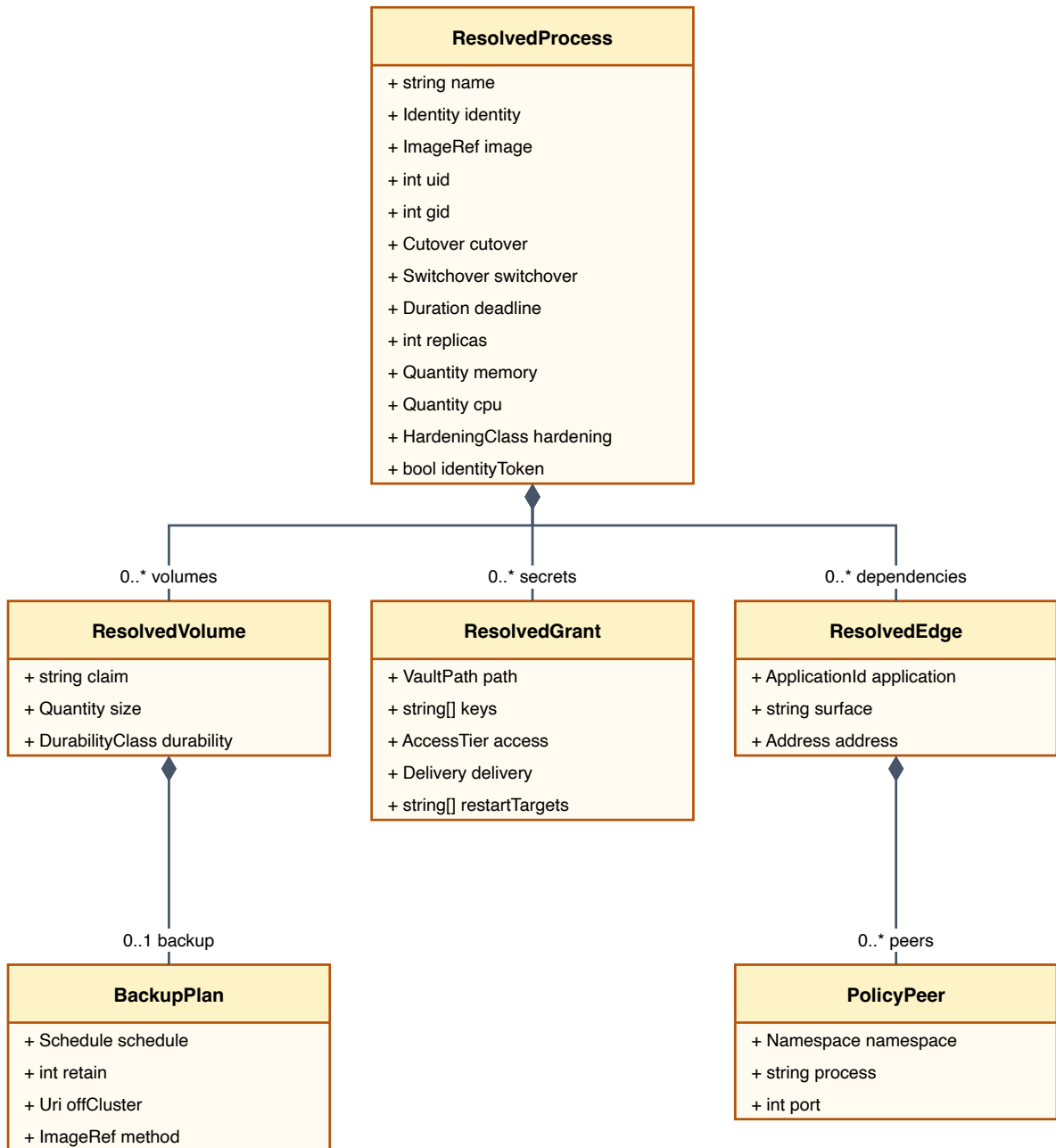


Figure 13: A Resolved Process: Storage, Secrets and Dependencies

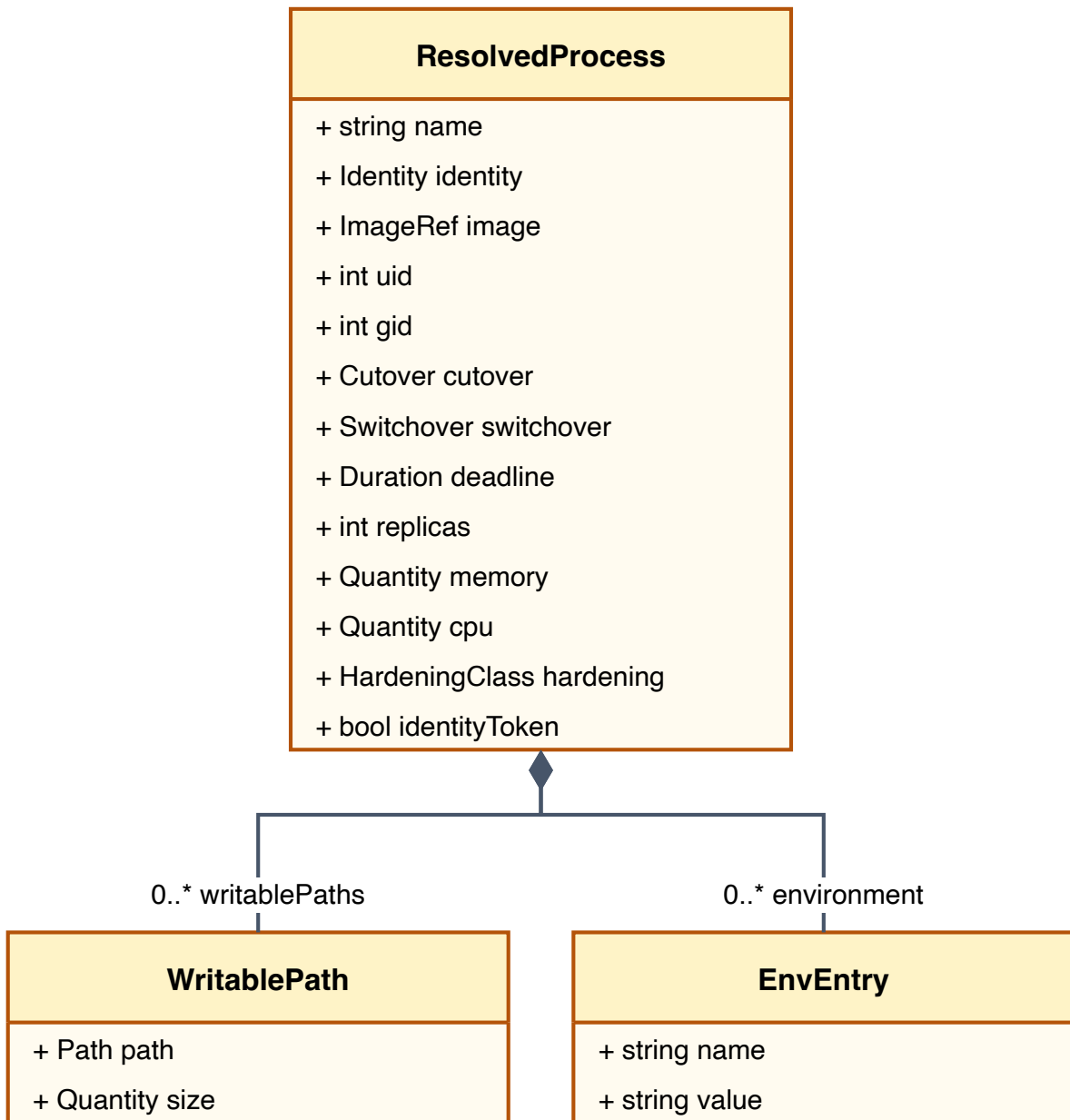


Figure 14: A Resolved Process: Writable Paths and Configuration

6.5 No Target Field Names

The constraint of Section 3 applies most visibly here. The resolved model records that a process switches blue/green; it does not record a rollout strategy or a Canary: the typed resource a generator walks for it is a `SwitchoverFile`, and only the generator spells it as a Canary. It records the hardening posture and the paths the process must write; it does not record a security context. It records the middleware chain as a sequence of named steps; it does not record the target proxy's spelling of them. Each generator is the one place its own target vocabulary appears, and that separation is checked directly rather than left to review.

7 The Concrete Syntax and the Constraints

The metamodels of Section 5 and Section 6 define what can be said. Two things are needed before a model can be written down and trusted: a concrete syntax an author types, and the constraints that refuse a model the abstract syntax alone would admit.

7.1 The Concrete Syntax

The source language has a textual concrete syntax, defined by an Xtext grammar over the metamodel rather than the other way round (Section 4). There are two grammars, one per authored document, and both import the registered metamodel package and return its classes: a rule never declares a type of its own, so no concept can enter the language through the syntax without existing in the metamodel first.

The syntax is the YAML subset the estate's files already use, which keeps the authored documents readable by the tools that read them today. Three properties of the grammar are worth reporting, because each is a consequence of that choice.

Indentation is tokenised, not parsed. Block structure reaches the parser as synthetic `BEGIN` and `END` tokens produced ahead of it, so an indented block, a flow mapping and a flow sequence all parse through one rule rather than three.

Keys are unordered. Every rule is an unordered group, because the order of keys in a mapping carries no meaning. Writing the rules as sequences would make the grammar refuse documents the model accepts.

Names are references. Where a route names the process and the surface it serves, the grammar declares a cross-reference rather than a string, so the name is resolved by the linker against the declared objects. A name with no target is a link that does not resolve, which is why a scrape naming an undeclared process (`E_UNKNOWN_PROCESS`), a tier naming an undeclared proxy (`E_UNKNOWN_TIER_PROXY`) and delivery machinery naming an undeclared application (`E_UNKNOWN_MACHINERY`) are not invariants in Table 5: the linker has already found them. The handover ledger's project names are the exception, and are plain strings: a project is the document itself, not an object another document can link to, so the ledger is checked by an invariant instead.

The editor is the generated Xtext one, with the constraints below loaded into it: the Complete OCL document binds to the same registered package the parser instantiates, so an invariant is evaluated against the objects the editor has just built, and a violation is reported in the editor rather than at render time.

7.2 The Constraints

Thirty-six invariants are written in Complete OCL against the source metamodel, carrying twenty-eight distinct diagnostic codes. Three codes sit on more than one context, because the same defect is reachable from several classes: a restated shared declaration is checked on each of the seven classes that can be restated, and two others on two classes each.

The document follows one naming rule, which is what makes a diagnostic self-locating: **an invariant is named by the code it emits, and its context is the object the diagnostic points at**. A violation is therefore a `(code, path)` pair with no lookup table in between, where the path is the RFC 6901 JSON Pointer of the refused object.

The last ten read facts from another document while sitting on a class of a project document: the platform document, or, for the three migration codes that ask whether a provider owns a database, the project file declaring that provider. That placement is deliberate: an invariant sits on the object whose author must change something, and it is a project file, not the platform, that has to be edited when a volume asks for a guarantee the estate does not keep. They reach the other document through `allInstances()`, whose extent is

Code	Context	What it refuses
E_ALERT_CLASS_WITHOUT_SIGNAL	Observability	A class stating how loudly to wake someone, with no signal to wake them about.
E_CUTOVER_UNHONOURABLE	Process	A continuous cutover over storage that cannot attach to the second copy a blue/green switchover starts.
E_CUTOVER_MISSING	Process	A process no level answers the cutover question for; a prepare process is exempt, because it cuts over nothing.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	An application whose processes answer the cutover question differently, when it switches as one.
E_PREPARE_PROCESS_SERVES	Process	A prepare process declaring a port, probes, replicas or a cutover, which only a serving process has.
E_SHARED_DECLARATION_DUPLICATED	seven classes	A shared declaration restated unchanged at a lower level, where it changes nothing.
E_SHARED_QUANTITY	Placement	A memory or cpu quantity above a process, which means something different for each process holding it.
E_PLACEMENT_INCOMPLETE	Process	A process with no quantity of its own, which nothing above it may supply.
E_OWNER_ROLE_GRANTED	DatabaseGrant	The role that changes the schema, granted by hand; it is derived for the migration alone.
E_MIGRATION_OWNER_DUPLICATED	Migration	A second application of one project declaring how the project's database schema moves.
E_ENGINE_WITHOUT_DURABILITY	Process	A backup method named where no volume's durability class asks for one.
E_DURABILITY_WITHOUT_ENGINE	Volume	A volume asking for a backup whose method would have to be guessed.
E_ENV_CANNOT_RELOAD	Rotation	A secret that tolerates a reload, delivered as an environment variable, which is read once at start.
E_ILLEGAL_DELIVERY_FOR_ACCESS	KvGrant	A delivery the access tier has nothing to project at render time.
E_NON_KV_DELIVERY	TransitGrant, DatabaseGrant	A grant delivered as a value, where the credential is used or minted rather than read.
E_DUPLICATE_ROUTE_MATCH	Route	Two routes of one exposure sharing a path and a match, which derived precedence cannot order.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	A tier serving authenticated visitors with nothing named to decide whether one is allowed in.
E_HANDOVER_BOTH_PATHS	HandoverLedger	A project named on both delivery paths at once, which two sources would each apply.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	An audience no declared edge tier serves, so the route reaches no edge.
E_NO_ENGINE_POLICY	Process	An engine the platform declares no backup method for.
E_NO_DURABILITY_POLICY	Volume	A durability class the platform declares no policy for.
E_SECRETS_AT_REST_REQUIRED	Grant	A secret written into a cluster that does not encrypt secrets at rest.
E_MIGRATION_UNDECLARED	Application	An application that reaches a database and says nothing about how its schema moves.
E_MIGRATION_WITHOUT_DATABASE	Migration	A migration on an application that reaches no database, decided only where every provider it reaches was read.
E_CREDENTIALS_WITHOUT_DATABASE	Credentials	A rotation override on an edge to a provider that owns no database.
E_NO_MIGRATION_POLICY	Migration	A changelog read beside a platform document that offers no runner to build it on.
E_NO_DELIVERY_POLICY	Application	A continuous application read beside a platform document that offers no cadence to analyse its blue/green switch at.
E_HANDOVER_UNLISTED	Project	A project read beside a handover ledger that names it on neither delivery path.

Table 5: The Constraints, Their Context and What Each Refuses

the resource set the document was read into, and each is guarded so that it holds trivially when a project file is read on its own:

```
context Exposure
  inv E_NO_TIER_FOR_AUDIENCE:
    Tier.allInstances()->notEmpty() implies
      Tier.allInstances()->exists(audiences->includes(self.audience))
```

Example 1: E_NO_TIER_FOR_AUDIENCE, guarded so a project file read alone still validates

One modelling detail shows up in several invariants and is worth naming, because it is a property of EMF rather than of the domain. EMF reads an unset enumeration attribute as its first literal, so an optional enumeration cannot be distinguished from one written with its first value. Each such vocabulary therefore carries a literal of its own for absence, **absent**, which a document cannot write, and the invariants test against it rather than against **null**.

Constraints that need every project document at once, such as two applications claiming one identifier, are not in this document. They belong to the composition step, where the union of the authored documents exists, and they are checked there against the fixtures in the negative set.

7.3 Every Constraint Has a Fixture

A constraint that has only ever been satisfied is not evidence that it fires. Each code in Table 5 is therefore paired with a committed model that violates it, and with a committed oracle file naming the set of (**code**, **path**) pairs that model must produce. The cross-document codes are fixtures of two files, a project document and the platform document it is read beside, because one file alone cannot express the defect.

Each fixture is named after its code, lower-cased and hyphenated, under `spec/v1/examples/refusals/`: E_NO_DELIVERY_POLICY is proved by `no-delivery-policy`. Two are named after the shape they refuse instead: `cutover-continuous-over-rwo` proves E_CUTOVER_UNHONOURABLE, and `cutover-mixed` proves E_RELEASE_UNIT_MIXED_CUTOVER.

The pairing is checked rather than asserted: each refused model is run through the parser and validated, and the diagnostics it produces are compared with the oracle. The accepted examples of Section 8 prove the language can express the estate; these prove it refuses what it says it refuses.

7.4 The Second Implementation

The same source language is declared a second time, by hand, in the TypeScript production implementation this repository exists to build. Neither declaration is generated from the other, and neither is the oracle for the other: each is tested on its own against the committed oracle files, and agreement between the two follows from both agreeing with the oracle.

They are compared on the four things the project proposal names: validation, so the parsed model of each accepted example matches one committed canonical JSON document; dependency resolution, so the resolved edges match a second; errors, so each refused model produces the same set of (**code**, **document**, **path**) triples; and generated resources, so the generated tree matches the committed one byte for byte. They are required to agree on nothing else, and in one respect they deliberately differ: the production implementation keeps the resolved model and the typed target objects apart, while the model-driven implementation holds both in the one target metamodel of Section 6, because a model-to-text template reads one model.

Two independent declarations of one language, held to shared oracles, is also what makes the metamodel falsifiable in a way a single implementation cannot be: a concept that only one of them can express is a concept the model has not actually pinned down.

8 Example Models

Five example models instantiate the source metamodel, and each is chosen to exercise something the metamodel claims. They are committed beside the metamodels, together with the expected result of parsing each one, so that the examples are checked rather than illustrative.

Every example passes through three stages, and each stage is a model or a file set of its own: the *input*, an authored project file together with the same model as an XMI instance of the source metamodel; the state

Example	What it exercises
minimal	The smallest complete model: one project, one application, one process, and nothing that is not required. Everything absent from it is either derived or refused, which makes it the model that shows what the language forces an author to say.
auth	Two processes in one application, both continuous, so the release gate has more than one member and the gate's deadline is a maximum over members rather than a single value; and a managed migration, because the application reaches a database.
data	Several applications in one project sharing a namespace, with storage and the backup plans their durability classes earn.
knowledge	Two applications in one project because their cutovers differ: a continuous API that moves the project's schema with a changelog, and an interrupted worker bound to a volume that declares no migration of its own. Dependencies across applications, secrets shared at the project header, and an exposure whose routes carry different audiences.
delivery	The platform's own delivery machinery, the Canary controller and the release gate: continuous, and rolled in place rather than switched blue/green, because a gate cannot gate its own release.

Table 6: The Example Models and What Each One Exercises

in between, the resolved model, an instance of the target metamodel; and the *output*, the generated file tree. Appendix A shows three of them through all three stages, cut down to what each is meant to show: the minimal model end to end (Appendix A.1), the **knowledge** project's two cutovers (Appendix A.2) and one refused model (Appendix A.3).

Where the files are. The metamodels are `emf/bundles/metamodel/model/project-intent.ecore` and `resolved-deployment.ecore`, with the constraints in `project-intent.oc1`. The examples in concrete syntax are under `spec/v1/examples/`, and the pipeline writes each one's XMI instance beside its parsed form on every build. The hand-written resolved model is `emf/models/minimal.resolveddeployment`. All of them, together with the generated trees and the refused set, are collected in the hand-in archive, whose layout is Appendix A.4. The concrete syntax is how the models are authored; the XMI files are the same models as Ecore instances, for a reader who opens them in Eclipse without the grammar.

8.1 The Minimal Model

The minimal model is the one to read first. It declares a project with one application and one process, and it is shorter than the file the same deployment needs today by roughly an order of magnitude. What it does *not* declare is the point: no rollout strategy, no resource limits, no security context, no labels, no selector, no namespace, no probe timings, no instance count. Each of those either derives from something the model does declare, or is platform policy, or is refused outright.

8.2 Refused Models

Alongside the valid examples, a set of deliberately invalid models is committed, one per constraint, each with the diagnostic it must produce. A constraint that has only ever been satisfied is not evidence that it fires, so each constraint is paired with a model that violates it.

The refused set covers, among others: a route naming a surface its process does not offer; two routes on one hostname claiming the same path; an alert class with nothing to collect metrics from; a process declaring a cutover its storage cannot honour; an application whose processes answer the cutover question differently; a prepare process that declares a port; an application that reaches a database and does not say how its schema moves, and one that says so without reaching one; a secret delivery that the cluster's encryption setting forbids; a durability class with no engine to back it up; and an exposure whose audience no edge tier serves.

8.3 A Resolved Model as a Deployment View

Figure 15 shows the **knowledge** example's resolved model the way a deployment diagram would draw it: the project's two processes, now two applications, placed on two machines, the volume one of them is bound to, the edge tier and hostname in front, and the two platform services they depend on. Nothing in the picture is new information. Every box and every arrow is read straight out of the resolved model, which is the point: the model is complete enough that a deployment view is a rendering of it rather than a separate document

somebody keeps in step by hand.

This is the part of the Task 0 feedback that is worth acting on. A structural notation is the right one for an arrangement, and generating this view from the target metamodel is another model-to-text transformation, no different in kind from the one that generates the deployment files.

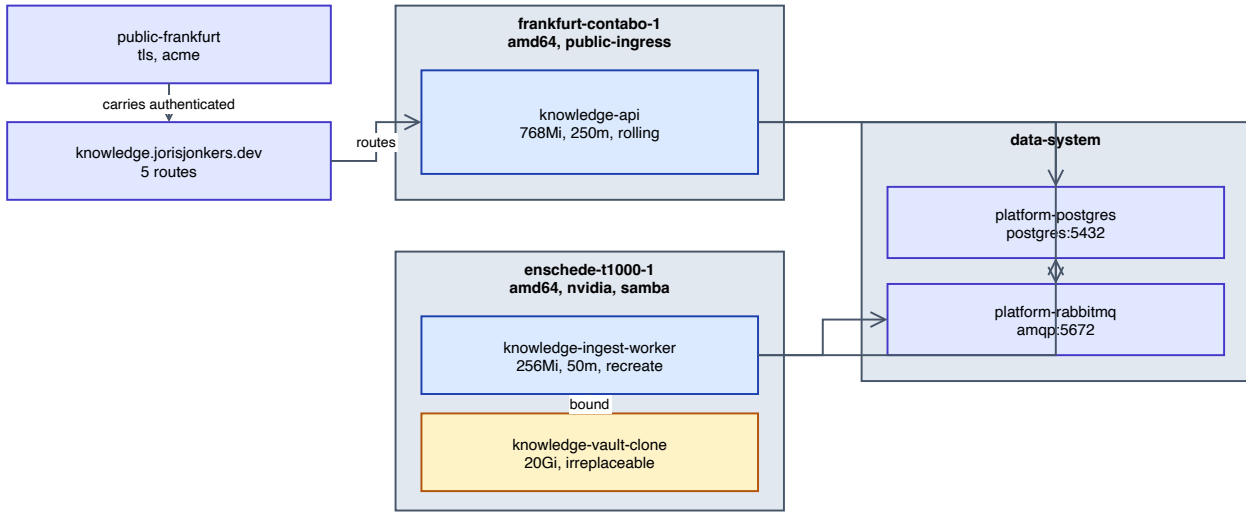


Figure 15: The knowledge Example's Resolved Model, as a Deployment View

8.4 Example Target Models

A hand-written target model accompanies the minimal example: the resolved model that the Task 2 transformation must produce from it, written and reviewed by hand rather than generated. It is the expected output of the transformation and the input the Task 3 templates are first run against, which is what lets both tasks be checked before either is complete.

A Worked Examples

Three examples, each shown as its input, its state in between and its output, cut down to what the example is meant to show. The complete files, and every other example, are in the hand-in archive (Appendix A.4).

A.1 The Minimal Model, End to End

The input is one project file in the concrete syntax of Section 7. Comments are removed here; the committed file carries one per decision.

```
apiVersion: intent.jorisjonkers.dev/v1
kind: Project
schemaVersion: 1.0.0
project: notes
owner: joris
applications:
- id: notes
  observability:
    alertClass: business-hours
    scrape: { process: notes-api, surface: http, path: /metrics }
  exposure:
    - name: public
      host: notes.jorisjonkers.dev
      audience: anonymous
      contentPolicy: strict
      routes:
        - { path: /, match: prefix, process: notes-api, surface: http }
  processes:
    - name: notes-api
      lifecycle: application
      image: notes-api
      runtime: node
      provides: { http: 8080 }
      placement: { memory: 256Mi, cpu: 50m }
      probes:
        readiness: { path: /healthz/ready, port: 8080 }
        liveness: { path: /healthz/live, port: 8080 }
      startupBudget: 20s
      cutover: continuous
```

Example 2: Input: minimal/notes.project.yml

The parser turns it into an instance of the source metamodel. The same instance, serialised as XMI, is what Eclipse opens without the grammar; attributes that hold their default value, such as `lifecycle: application`, are not written.

```
<projectintent:Project apiVersion="intent.jorisjonkers.dev/v1" kind="Project"
  schemaVersion="1.0.0" project="notes" owner="joris">
  <applications id="notes">
    <observability>
      <scrape process="notes-api" surface="http" path="/metrics"/>
    </observability>
    <exposure name="public" host="notes.jorisjonkers.dev" audience="anonymous" contentPolicy="strict">
      <routes path="/" process="notes-api" surface="http"/>
    </exposure>
    <processes startupBudget="20s" cutover="continuous" name="notes-api" image="notes-api" runtime="node">
      <placement memory="256Mi" cpu="50m"/>
      <provides key="http" value="8080"/>
      <probes xsi:type="projectintent:Probes">
        <readiness xsi:type="projectintent:HttpProbe" path="/healthz/ready" port="8080"/>
        <liveness xsi:type="projectintent:HttpProbe" path="/healthz/live" port="8080"/>
      </probes>
    </processes>
  </applications>
</projectintent:Project>
```

Example 3: Input as a model: minimal/intent.xmi

In between is the resolved model: an instance of the target metamodel, written by hand as the result the Task 2 transformation must produce. Every value in it is either carried from the input or decided by the platform, and nothing in it names a Kubernetes field.

```
<applications id="notes" revision="sha256:..." project="notes"
  namespace="notes-system" alertClass="business-hours" reconcileUnit="//@reconcileUnits.0">
  <releaseGate deadline="60s">
    <analysis interval="30s" iterations="4" threshold="3"/>
    <members process="//@applications.0/@processes.0">
      <readiness xsi:type="resolveddeployment:HttpTarget" path="/healthz/ready" port="8080"/>
```

```

    <checks>error-rate</checks>
    <checks>latency</checks>
  </members>
</releaseGate>
<exposure name="public" host="notes.jorisjonkers.dev" tier="public-frankfurt">
  <routes path="/" match="prefix" audience="anonymous" precedence="1">
    process="//applications.0/@processes.0" surface="http">
      <middleware kind="security-headers" contentPolicy="strict"/>
    </routes>
  </exposure>
<processes name="notes-api" identity="notes-api" image="ghcr.io/.../notes-api@sha256:9c1e..."
  uid="1000" gid="1000" cutover="continuous" switchover="blue-green" deadline="60s"
  replicas="1" memory="256Mi" cpu="50m" hardening="restricted" identityToken="false">
  <startup period="5s" failures="4">
    <target xsi:type="resolveddeployment:HttpTarget" path="/healthz/live" port="8080"/>
  </startup>
  <placement eligibleNodes="enschede-t1000-1 ... frankfurt-contabo-1"/>
</processes>
</applications>

```

Example 4: In between: `minimal.resolveddeployment`, the application

Table 7 reads the two side by side: what the author wrote, and what the platform decided from it.

Authored	Decided in the resolved model
project: notes	namespace="notes-system", and the reconcile unit
cutover: continuous	switchover="blue-green", and a <code>releaseGate</code> with this process as its one member
startupBudget: 20s	a startup probe of 4 checks 5 seconds apart, and a deadline of 60 seconds
runtime: node	the checks <code>error-rate</code> and <code>latency</code> , because the profile exposes HTTP server metrics
image: notes-api	the image digest, and the numeric user and group it runs as
audience: anonymous	the edge tier <code>public-frankfurt</code> , and a middleware chain without forward authentication
placement	the machines the process is eligible for: all seven, because nothing narrows the set

Table 7: The Minimal Model: What Was Authored, and What Was Decided

The output is the generated file tree: nine files for this one process. The Canary is the object through which the process is switched; every decision in it was already in the resolved model, and only the generator spells it.

```

apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: notes-api
  namespace: notes-system
spec:
  provider: kubernetes
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: notes-api }
  progressDeadlineSeconds: 60
  service: { port: 8080, targetPort: http, portName: http }
  analysis:
    interval: 30s
    iterations: 4
    threshold: 3
    webhooks:
      - name: may-promote
        type: confirm-promotion
        url: http://release-gate.delivery-system.svc.cluster.local:8080/may-promote
        metadata: { application: notes, process: notes-api, revision: "sha256:..." }

```

Example 5: Output: `minimal/rendered/apps/notes/canary.yaml`, one webhook of three

A.2 Two Cutovers in One Project

The knowledge example shows how one authored question changes the shape of the output. Its two applications answer the cutover question differently, so they are two applications, and only one of them moves the project's schema.

```

applications:
  - id: knowledge
    migration:
      changelog: knowledge-api/src/main/resources/db/changelog.yml
    processes:
      - name: knowledge-api

```

```

    runtime: jvm
    startupBudget: 600s
    cutover: continuous
- id: knowledge-ingest
  migration: none
  processes:
  - name: knowledge-ingest-worker
    runtime: python
    probes: none
    cutover: interrupted
    volumes:
    - { claim: knowledge-vault-clone, size: 20Gi, durability: irreplaceable }

```

Example 6: Input: knowledge/knowledge.project.yml, abridged

In between, the continuous application carries a release gate and a migration; the interrupted one carries neither, because it stops before it starts and nothing waits on a gate.

```

knowledge:
  "migration": { "runner": ".../knowledge-migration@sha256:c0a4...",
                 "testedAgainst": "sha256:5a1f...", "nonTransactional": false },
  "releaseGate": { "deadline": "1800s",
                   "members": [ { "process": "knowledge-api",
                                   "checks": ["error-rate", "latency"] } ] },
  "processes": [ { "name": "knowledge-api", "switchover": "blue-green" } ]

knowledge-ingest:
  "processes": [ { "name": "knowledge-ingest-worker", "cutover": "interrupted",
                  "switchover": "stop-start" } ]

```

Example 7: In between: the two applications' projections, abridged

In the output, the migration becomes two Jobs named for the revision: the one that moves the schema, and the one that would undo it, rendered suspended so that only the release gate can start it.

```

apiVersion: batch/v1
kind: Job
metadata:
  name: knowledge-migration-down-6325500a2c54
  annotations:
    kustomize.toolkit.fluxcd.io/ssa: IfNotPresent
spec:
  suspend: true
  backoffLimit: 0
  template:
    spec:
      serviceAccountName: knowledge-migration
      containers:
      - name: migration
        image: ghcr.io/jorisjonkers-dev/knowledge/knowledge-migration@sha256:c0a4...
        args: [down, --to=sha256:5a1f...]

```

Example 8: Output: knowledge/rendered/apps/knowledge/migration.yaml, the down, abridged

A.3 A Refused Model

A prepare process runs once before the new version starts, so it serves nothing. Declaring what only a serving process has is refused, and the diagnostic names the object at fault by its JSON Pointer.

```

- name: seed
  lifecycle: prepare
  image: seed
  runtime: none
  placement: { memory: 64Mi, cpu: 10m }
  startupBudget: 60s
  provides: { http: 8080 }
  probes:
    readiness: { path: /ready, port: 8080 }
  replicas: { count: 2, reason: A step that runs once has no second copy to spread. }
  cutover: continuous

```

Example 9: Input: refusals/prepare-process-serves.project.yml, the refused process

```

[{"code": "E_PREPARE_PROCESS_SERVES", "document": "prepare-process-serves.project.yml",
 "path": "/applications/0/processes/1"}]

```

Example 10: Output: refusals/prepare-process-serves.diagnostics.json

A.4 The Hand-in Archive

The archive handed in with this report holds the metamodels and every model referred to here. Table 8 lists its layout. Each example has the same three stages as the examples above. The resolved model exists as XMI for the minimal example and as the production implementation’s JSON projection for `minimal`, `knowledge` and `delivery`; the others have none yet, because producing it is the Task 2 transformation.

Path	What it holds
<code>metamodels/</code>	<code>project-intent.ecore</code> , <code>resolved-deployment.ecore</code> , and <code>project-intent.ocl</code> , the constraints of Section 7
<code>examples/<name>/1-input/</code>	the authored files in concrete syntax, and <code>intent.xmi</code> , the same model as an instance of the source metamodel
<code>examples/<name>/2-resolved/</code>	the resolved model: <code>minimal.resolveddeployment</code> as XMI, and <code>resolved*.json</code> projections
<code>examples/<name>/3-output/</code>	the generated file tree
<code>examples/platform/</code>	the Platform document, and its model as XMI
<code>refusals/</code>	every refused model with the diagnostics it must produce

Table 8: The Layout of the Hand-in Archive